VIPER Programming Language

Complete Language Specification — Lexemes, Tokens, Classes & Regular Expressions

Design principle: VIPER uses snake- and viper-inspired terminology for its language constructs. The themed term is used in the language syntax, while the original programming meaning is shown in parentheses.

1. Program Beginning and End

Formal Language: A VIPER source program is bounded by a beginning marker and an ending marker.

VIPER Term	Original Semantics	Class
HOLD	program start	Hold
RELEASE	program end	Release

HOLD (program start)

Meaning: `HOLD` marks the beginning of a VIPER source program.

Class: Hold

RE: ^HOLD$

Explanation: `^` anchors the match at the beginning, the keyword must match exactly, and `$` anchors the match at the end.

Example: HOLD

RELEASE (program end)

Meaning: `RELEASE` marks the end of a VIPER source program.

Class: Release

RE: ^RELEASE$

Explanation: `^` anchors the match at the beginning, the keyword must match exactly, and `$` anchors the match at the end.

Example: RELEASE

Basic structure: HOLD ... RELEASE

1.1 Single-Line Comments

Formal Language: A single-line comment is used to add explanatory or descriptive text that is ignored by the lexical analyzer. It begins with the # symbol and continues until the end of the current line.

Class: COMMENT

RE:

^#.*$

Explanation of RE

- ^ — marks the beginning of the comment.
- # — identifies the beginning of a single-line comment.
- .* — allows zero or more characters after #.
- $ — marks the end of the current line.

Examples

Valid:

This is a comment
fang age = 20; # stores the age
VIPER single-line comment

An inline comment:

fang age = 20; # age of student

The lexer processes:

fang → DT
age → ID
= → ASSIGN_OP
20 → DIGIT
; → SYMBOL

and ignores:

age of student

Invalid / Special Cases

fang age = 20 #

This is **still a valid single-line comment** because everything after # on that line is treated as a comment.

#

This is also valid because `.*` allows zero characters after `#`.

Lexical Behavior

Once the lexer encounters `#` outside a string or character literal:

——————————————► ignore everything until newline

The newline itself is not part of the comment and should still be processed so that the lexer can correctly maintain line numbers.

1.2 Multi-Line Comments

Formal Language: A multi-line comment is used to write comments spanning multiple lines. It begins with `#` and ends with the next `#` symbol.

Class: `MULTILINE_COMMENT`

RE:

^#(.|\n)*?#$

Explanation of RE

- `^` — marks the beginning of the comment.
- `#` — identifies the opening delimiter.
- `(.|\n)` — allows any character, including newline characters.
- `*?` — allows zero or more characters while stopping at the first possible closing `#`.
- `#` — identifies the closing delimiter.
- `$` — marks the end of the complete multiline comment.

Valid Examples

```
# This is a
multi-line comment #
#
This is a VIPER
multi-line comment
#
fang age = 20;

# This section calculates
the student's age
```

and stores it #

hiss ready = true;

2. VIPER Data Types

Formal Language: VIPER provides data types for whole numbers, decimal values, characters, strings, Boolean values, and functions that return no value.

VIPER Term	Original Semantics	Class
fang	integer	DT
venom	float	DT
scale	character	DT
coil	string	DT
hiss	boolean	DT
shed	void	DT

fang (integer)

Formal Language: `fang` is the VIPER data-type keyword used to represent whole-number values.

Class: DT

RE: ^fang$

Explanation: The expression requires the exact keyword `fang`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: fang age = 20;

venom (float)

Formal Language: `venom` is the VIPER data-type keyword used to represent decimal/fractional values.

Class: DT

RE: ^venom$

Explanation: The expression requires the exact keyword `venom`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: venom price = 19.95;

scale (character)

Formal Language: `scale` is the VIPER data-type keyword used to represent exactly one character.

Class: DT

RE: ^scale$

Explanation: The expression requires the exact keyword `scale`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: scale grade = 'A';

coil (string)

Formal Language: `coil` is the VIPER data-type keyword used to represent a sequence of characters.

Class: DT

RE: ^coil$

Explanation: The expression requires the exact keyword `coil`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: coil name = "Fahad";

hiss (boolean)

Formal Language: `hiss` is the VIPER data-type keyword used to represent a true/false logical state.

Class: DT

RE: ^hiss$

Explanation: The expression requires the exact keyword `hiss`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: hiss ready = true;

shed (void)

Formal Language: `shed` is the VIPER data-type keyword used to represent the absence of a returned value.

Class: DT

RE: ^shed$

Explanation: The expression requires the exact keyword `shed`. `^` marks the start and `$` marks the end, so additional characters are not accepted.

Example: nest shed display() { ... }

3. Identifiers

Formal Language: An identifier is a programmer-defined name used to identify variables and other named program elements. It begins with a letter or underscore and may be followed by letters, digits, or underscores.

Class: ID

RE: ^[a-zA-Z_][a-zA-Z0-9_]*$

- `^` — beginning of the identifier.
- `[a-zA-Z_]` — first character is a letter or underscore.
- `[a-zA-Z0-9_]*` — zero or more letters, digits, or underscores may follow.
- `$` — end of the identifier.

Examples: x, age, student1, _count, totalMarks, StudentRecord

Status	**Example**	**Reason**
Valid	x	Starts with a letter.
Valid	_count	Starts with an underscore.
Valid	student1	Digits are allowed after the first character.
Invalid	2age	Cannot begin with a digit.
Invalid	user-name	Hyphen is not permitted.
Invalid	total marks	Spaces are not permitted.
Invalid	@value	@ is not permitted.

Reserved-word rule: VIPER keywords such as `fang`, `strike`, and `nest` cannot be used as ordinary identifiers.

4. Initialization

Formal Language: Initialization is the process of assigning an initial value to a variable at the time the variable is created. In VIPER, initialization combines a data type, an identifier, an assignment operator, and a valid literal or expression.

Initialization Syntax

data_type identifier = value;

Where:

- data_type — one of the VIPER data types such as fang, venom, scale, coil, or hiss.
- identifier — the name of the variable.
- = — the assignment operator.
- value — the initial value assigned to the variable.
- ; — terminates the statement.

Initialization Structure

fang age = 20;

venom price = 19.95;

scale grade = 'A';

coil name = "Fahad";

hiss ready = true;

Initialization Class

Class: Initialization (INIT)

RE

^(fang|venom|scale|coil|hiss)\s+[a-zA-Z_][a-zA-Z0-9_]*\s*=\s*.+;$

Explanation of RE

- ^ — marks the beginning of the statement.
- (fang|venom|scale|coil|hiss) — accepts one of the VIPER data-type keywords.
- \s+ — requires one or more spaces between the data type and identifier.
- [a-zA-Z_] — requires the identifier to begin with a letter or underscore.
- [a-zA-Z0-9_]* — allows letters, digits, and underscores after the first character.
- \s* — allows zero or more spaces before and after the assignment operator.
- = — represents initialization/assignment.
- .+ — requires at least one value or expression.

· ; — requires the statement terminator.

· $ — marks the end of the complete statement.

Examples

```
fang age = 20;

venom temperature = 36.5;

scale grade = 'A';

coil studentName = "Fahad";

hiss enrolled = true;
```

Valid Examples

Example	Reason
fang age = 20;	Valid integer initialization
venom price = 19.95;	Valid float initialization
scale grade = 'A';	Valid character initialization
coil name = "Fahad";	Valid string initialization
hiss ready = true;	Valid Boolean initialization

Invalid Examples

Example	Reason
fang = 20;	Identifier is missing
fang 123age = 20;	Identifier cannot begin with a digit
venom price = ;	No value is provided
scale grade = 'AB';	Character literal contains more than one character

hiss ready = 10;	Incorrect value for a Boolean variable
coil name = Fahad;	String value requires quotation marks

VIPER Initialization Examples by Data Type

fang (integer)

fang age = 20;

venom (float)

venom height = 5.9;

scale (character)

scale section = 'A';

coil (string)

coil username = "Fahad";

hiss (boolean)

hiss active = true;

---

5. Declaration

Formal Language: Declaration is the process of introducing a variable by specifying its data type and identifier without assigning an initial value. The variable is made known to the program, but its value is assigned later.

Declaration Syntax

data_type identifier;

Where:

- data_type — specifies the type of data the variable can store.
- identifier — specifies the variable's name.
- ; — terminates the declaration statement.

Declaration Structure

fang age;

venom price;

scale grade;

coil name;

hiss ready;

Declaration Class

Class: Declaration (DEC)

RE

^(fang|venom|scale|coil|hiss)\s+[a-zA-Z_][a-zA-Z0-9_]*\s*;$

Explanation of RE

- ^ — marks the beginning of the declaration.
- (fang|venom|scale|coil|hiss) — accepts one of the VIPER data-type keywords.
- \s+ — requires one or more spaces between the data type and identifier.
- [a-zA-Z_] — requires the identifier to begin with a letter or underscore.
- [a-zA-Z0-9_]* — permits letters, digits, and underscores after the first character.
- \s* — permits optional spaces before the semicolon.
- ; — identifies the end of the declaration.
- $ — marks the end of the complete statement.

Examples

fang age;

venom price;

scale grade;

coil name;

hiss ready;

Valid Examples

Example	Reason
fang age;	Valid integer declaration
venom price;	Valid float declaration

scale grade;	Valid character declaration
coil name;	Valid string declaration
hiss ready;	Valid Boolean declaration
fang student_age;	Valid identifier
coil student1;	Valid identifier containing a digit

Invalid Examples

Example	Reason
fang;	Identifier is missing
fang 123age;	Identifier cannot begin with a digit
fang student-age;	Hyphen is not allowed in an identifier
fang age = 20;	This is initialization, not declaration
20 age;	A data-type keyword is required
coil student name;	Spaces are not allowed inside an identifier

Declaration Followed by Assignment

A variable can first be declared and then initialized later.

```
fang age;
age = 20;
```

Another example:

```
coil name;
name = "Fahad";
```

Here, the first statement is a **declaration**, while the second statement is an **assignment**.

Declaration vs Initialization

Feature	Declaration	Initialization
Data type specified	Yes	Yes
Identifier specified	Yes	Yes
Initial value provided	No	Yes
Example	fang age;	fang age = 20;
Class	DEC	INIT

Combined Syntax

Declaration:

data_type identifier;

Initialization:

data_type identifier = value;

So the distinction in VIPER is:

fang age; // Declaration

fang marks = 95; // Initialization

6. Literals

Formal Language: A literal is a fixed value written directly in VIPER source code.

Literal Type	Example	Class

Integer	7	Fang
Float	3.14	Venom
Character	'A'	Scale
String	"Hello"	Coil
Boolean	true	Hiss
Boolean	false	Hiss

6.1 Integer Literal (fang value)

Formal Language: An integer literal represents a whole-number value and is used as a value for a `fang (integer)` variable.

Class: Integer Literal

RE: ^[0-9]+$

Explanation: `^` begins the value; `[0-9]+` requires one or more digits; `$` ends the value.

Valid examples: 0, 7, 42, 13580

Invalid examples: 3.14, abc, 12a

6.2 Float Literal (venom value)

Formal Language: A float literal represents a decimal value and is used as a value for a `venom (float)` variable.

Token: FLOAT_CONST

Class: Float Literal

RE: ^[0-9]+\.[0-9]+$

Explanation: The first digit group is before the decimal; `\.` matches the decimal point; the second digit group is after it.

Valid examples: 3.14, 0.5, 2.0

Invalid examples: 3, .5, 5.

6.3 Character Literal (scale value)

Formal Language: A character literal contains exactly one character inside single quotation marks.

Token: CHAR_CONST

Class: Character Literal

RE: ^'.'$

Explanation: The opening quote is required, `.` represents one character, and the closing quote is required.

Valid examples: 'A', 'z', '9'

Invalid examples: 'AB', A, "A"

6.4 String Literal (coil value)

Formal Language: A string literal is a sequence of characters enclosed in double quotation marks.

Token: STRING_CONST

Class: String Literal

RE: ^".*"$

Explanation: The opening double quote is required, `.*` permits zero or more characters, and the closing double quote is required.

Valid examples: "Hello", "VIPER", ""

Invalid examples: Hello, 'Hello', "Hello

6.5 Boolean Literal (hiss value)

Formal Language: A Boolean literal represents one of the two logical states and is used as a value for a `hiss (boolean)` variable.

Token: BOOL_CONST

Class: Boolean Literal

RE: ^(true|false)$

Explanation: `(true|false)` accepts exactly one of the two alternatives, with anchors requiring a complete match.

Valid examples: true, false

Invalid examples: True, FALSE, 1, yes

7. Assignment Operators

Lexeme	Class	Purpose
=	=	Simple assignment
+=	+=	Add and assign
-=	-=	Subtract and assign
*=	*=	Multiply and assign
/=	/=	Divide and assign

RE: ^(=|\+=|-=|*=|/=)$

Explanation: The alternatives represent the five assignment operators. `|` means OR and special regex symbols are escaped where needed.

Examples: fang x = 10; x += 2; x -= 1; x *= 3; x /= 2;

8. Arithmetic Operators

Lexeme	Class	Purpose
+	+	Addition
-	-	Subtraction
*	*	Multiplication
/	/	Division
%	%	Modulus

RE: ^(\+|-|*|/|%)$

Explanation: The grouped alternatives match one of the five arithmetic operators. `+` and `*` are escaped because they are regex metacharacters.

Examples: a + b, a - b, a * b, a / b, a % b

9. Relational Operators

Lexeme	Class	Meaning
==	==	Equal to

!=	!=	Not equal to
<	<	Less than
>	>	Greater than
<=	<=	Less than or equal to
>=	>=	Greater than or equal to

RE: ^(==|!=|<=|>=|<|>)$

Explanation: The expression accepts one of the six defined relational operators. Two-character operators are explicitly included.

Examples: age == 18, age != 18, age < 18, age > 18, age <= 18, age >= 18

10. Logical Operators

Lexeme	**Class**	**Meaning**
&&	&&	Logical AND
||	||	Logical OR
!	!	Logical NOT

RE: ^(&&|\|\||!)$

Explanation: The expression accepts `&&`, `||`, or `!`. The vertical bars of `||` are escaped because `|` is the regex OR operator.

Examples: a && b, a || b, !ready

11. Keywords

Formal Language: Keywords are reserved words with predefined lexical or syntactic meanings in VIPER and cannot be used as ordinary identifiers.

VIPER Keyword	Original Semantics	Class	Purpose
fang	integer	DT	Whole-number data type
venom	float	DT	Decimal data type
scale	character	DT	Single-character data type
coil	string	DT	String data type
hiss	boolean	DT	Boolean data type
shed	void	DT	No-return-value type
strike	if	Strike	Conditional branch
shift	else-if	Shift	Alternative conditional test
slither	else	Slither	Final alternative branch
coilrun	for loop	Coilrun	Controlled repetition
stalk	while loop	Stalk	Condition-controlled repetition
lurk	do-while loop	Lurk	Body-first repetition
escape	break	Escape	Exit nearest loop
slide	continue	Slide	Skip current iteration
nest	function	Nest	Function definition
fangback	return	Fangback	Return from function
sense	input	Sense	Input operation
hissout	output / print	Hissout	Output operation

General RE:
^(fang|venom|scale|coil|hiss|shed|strike|shift|slither|coilrun|stalk|lurk|escape|slide|nest|fangback|sense|hissout)$

Explanation: Each alternative is a reserved VIPER word. The anchors require the complete lexeme to match one of the listed keywords.

12. Special Symbols

Symbol	Class	Role
(	(	Opening parenthesis
)	)	Closing parenthesis
{	{	Opening block delimiter
}	}	Closing block delimiter
[	[	Opening bracket
]	]	Closing bracket
;	;	Statement terminator/separator
,	,	Separates items or arguments
:	:	Separates syntactic components

RE: ^(\(|\)|\{|\}|\[|\]|;|,|:)$

Explanation: Each alternative represents one literal special symbol. Parentheses and brackets are escaped because they have special meanings in regular expressions.

Examples: ( ), { }, [ ], ; , :

11. Regular Expression Reference

Lexical Element	Class	RE
HOLD	Hold	^HOLD$
RELEASE	Release	^RELEASE$
fang (integer)	DT	^fang$
venom (float)	DT	^venom$
scale (character)	DT	^scale$
coil (string)	DT	^coil$
hiss (boolean)	DT	^hiss$
shed (void)	DT	^shed$

Identifier	ID	^[a-zA-Z_][a-zA-Z0-9_]*$
Integer literal	DIGIT	^[0-9]+$
Float literal	FLOAT_CONST	^[0-9]+\.[0-9]+$
Character literal	CHAR_CONST	^'.'$
String literal	STRING_CONST	^".*"$
Boolean literal	BOOL_CONST	^(true|false)$
Assignment	ASSIGN_OP	^(=|\+=|-=|*=|/=)$
Arithmetic	ARITH_OP	^(\+|-|*|/|%)$
Relational	RELOP	^(==|!=|<=|>=|<|>)$
Logical	LOG_OP	^(&&|||!)$
Keywords	Above	See keyword list above
Special symbols	SYMBOL	^(\(|\)|\{|\}|\[|\];|,|:)$

12. Examples of VIPER Constructs

Data declarations: fang age = 20; venom price = 12.5; scale grade = 'A'; coil name = "VIPER"; hiss ready = true;

Conditional: strike(age >= 18) { hissout("Adult"); } shift(age >= 13) { hissout("Teen"); } slither { hissout("Child"); }

Loop: coilrun(i = 0; i < 5; i++) { hissout(i); }

Function: nest fang add(fang a, fang b) { fangback a + b; }

Input/Output: fang age = sense(); hissout(age);

Program: HOLD ... RELEASE

13. Consolidated Token / Class Table

Category	**Lexeme**	**Class**	**Meaning**
Program Boundary	HOLD	HOLD	Program Start
Program Boundary	RELEASE	RELEASE	Program End

Data Type	fang	DT	DT — integer
Data Type	venom	DT	DT — float
Data Type	scale	DT	DT — character
Data Type	coil	DT	DT — string
Data Type	hiss	DT	DT — boolean
Data Type	shed	DT	DT — void
Identifier	letter/_ + letters/digits/_	ID	Identifier
Integer Literal	digits	DIGIT	Integer Literal
Float Literal	digits.decimal digits	FLOAT_CONST	Float Literal
Character Literal	'character'	CHAR_CONST	Character Literal
String Literal	"characters"	STRING_CONST	String Literal
Boolean Literal	true / false	BOOL_CONST	Boolean Literal
Assignment	= += -= *= /=	ASSIGN_OP	Assignment Operator
Arithmetic	+ - * / %	ARITH_OP	Arithmetic Operator
Relational	== != < > <= >=	RELOP	Relational Operator
Logical	&& || !	LOG_OP	Logical Operator
Conditional	strike / shift / slither	RK	if / else-if / else
Loop	coilrun / stalk / lurk	RK	for / while / do-while
Loop Control	escape / slide	RK	break / continue
Function	nest / fangback	RK	function / return
Input / Output	sense / hissout	RK	input / output
Special Symbol	( ) { } [ ] ; , :	SYMBOL	Special Symbol

14. VIPER Naming Consistency

Rule: Every themed VIPER term has exactly one language meaning. No themed term is reused for a different construct.

· `fang` = integer only.

· `venom` = float only.

· `scale` = character only.

· `coil` = string only.

· `hiss` = boolean only.

· `shed` = void only.

· `strike`, `shift`, `slither` = conditional control only.

· `coilrun`, `stalk`, `lurk` = loop constructs only.

· `escape`, `slide` = loop-control operations only.

· `nest`, `fangback` = function-related constructs only.

· `sense`, `hissout` = input/output operations only.